OBJECT-ORIENTED PROGRAMMING

MIDTERM PRACTICE BOOKLET

Syllabus-Aligned Practice Problems (Direct Field Access Style)

Based on Basics, Circle, Rectangle & Flag Lecture Materials

Topics Covered:

- Classes, Objects, Instantiation & Reference Assigning (Direct Access)
- Constructors & Parameter Shadowing (this.x = x)
- Static Fields & Methods (Tracing counters & class-level variables)
- Geometric Composition (Point & Circle checks)
- Overlapping Composition (Circle inside Rectangle & Flag balances)
- Problem-Solving Scenarios without private encapsulation keywords

Prepared for UIU OOP CSE1115 Students

University Level Midterm Study Material

Chapter 1: Classes, Objects, and Variables (Direct Field Access)

In this chapter, all classes define fields without access specifiers like 'private'. Fields are accessed directly by objects (e.g. p1.x = 5) or within internal class methods, directly matching the coding style taught in your class notes.

Question: Trace the output of the following Java program (based on basics_OOP.txt):

```
class Point {
    int x, y;
    static int s;

    Point(int a, int b) {
        x = a;
        y = b;
        System.out.println("hello");
    }
}

public class Main {
    public static void main(String[] args) {
        Point p1 = new Point(1, 2);
        Point p2 = new Point(10, 10);
        System.out.println(p1.x);
        System.out.println(p2.x);
        System.out.println(p1.s);
        p1.s = 5;
        System.out.println(p2.s);
        Point.s = 7;
        System.out.println(p1.s);
        System.out.println(p1.x);
    }
}
```

Answer: Output Console:

hello
hello
1
10
0
5
7
1

Explanation:
- Creating p1 prints 'hello' and sets p1.x=1, p1.y=2.
- Creating p2 prints 'hello' and sets p2.x=10, p2.y=10.
- 'p1.x' prints 1. 'p2.x' prints 10.
- The static variable 's' is default-initialized to 0. So 'p1.s' prints 0.
- 'p1.s = 5' sets the shared static variable 's' to 5. So 'p2.s' now prints 5.
- 'Point.s = 7' modifies the static variable to 7. So 'p1.s' now prints 7.

- 'p1.x' remains 1 and is printed.

Chapter 2: The 'this' Keyword & Parameter Shadowing

When parameters have the same name as class variables (e.g. x, y), shadowing occurs. Inside the constructor, 'x' refers to the parameter. To store this value in the instance variable, we use 'this.x = x;'.

Question: Correct the shadowing bug in the following constructor using 'this':

```
class Point {
    int x, y;
    Point(int x, int y) {
        x = x;
        y = y;
    }
}
```

Answer: The assignments 'x = x' and 'y = y' assign the parameters to themselves. The instance variables remain at default value 0.
To correct this, use 'this' to refer to instance variables:
Point(int x, int y) {
this.x = x;
this.y = y;
}

Chapter 3: Composition & Direct Variable Referencing

Composition involves nesting objects within other objects (e.g., Circle has a Point 'center'). Without encapsulation, we access the fields of nested objects using direct dot notation: 'circle.center.x'.

Question: Given the classes Point and Circle, explain how to calculate the distance between the circle center and an arbitrary point p. Write the code without using any getters/setters.

```
class Point {
    int x, y;
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
    double distance(Point p) {
        int a = this.x;
        int b = this.y;
        int c = p.x;
        int d = p.y;
        return Math.sqrt((a - c) * (a - c) + (b - d) * (b - d));
    }
}

class Circle {
    Point center;
    double radius;
    Circle(Point center, double radius) {
        this.center = center;
        this.radius = radius;
    }
    void check(Point p) {
        double dist = p.distance(this.center);
        if (dist > this.radius) {
            System.out.println("outside");
        } else {
            System.out.println("inside");
        }
    }
}
```

Answer: The method 'check(Point p)' uses composition. It accesses 'this.center' (which is a Point object) and calls the Point class method 'p.distance(this.center)' to calculate distance. This operates completely with direct variable access, without using any getCenter() or getRadius() methods.

Chapter 4: Geometric Overlaps and Shape containment

Below are the implementations for Circle inside Circle, Circle inside Rectangle, and Rectangle inside Rectangle, and the balanced flag check, all formatted using direct variable access.

Question: Implement a method circleInsideCircle(Circle C) inside the Circle class using direct field access:

```
// Inside Circle class:
boolean circleInsideCircle(Circle C) {
    double centerDist = this.center.distance(C.center);
    return (centerDist + C.radius) <= this.radius;
}
```

Question: Implement circleInsideRectangle(Circle C) and RectangleInsideRectangle(Rectangle R) inside the Rectangle class:

```
class Rectangle {
    Point bottomLeft, topRight;
    Rectangle(Point bottomLeft, Point topRight) {
        this.bottomLeft = bottomLeft;
        this.topRight = topRight;
    }

    boolean circleInsideRectangle(Circle C) {
        double left = C.center.x - C.radius;
        double right = C.center.x + C.radius;
        double bottom = C.center.y - C.radius;
        double top = C.center.y + C.radius;

        return left >= this.bottomLeft.x &&
               right <= this.topRight.x &&
               bottom >= this.bottomLeft.y &&
               top <= this.topRight.y;
    }

    boolean RectangleInsideRectangle(Rectangle R) {
        return R.bottomLeft.x >= this.bottomLeft.x &&
               R.topRight.x <= this.topRight.x &&
               R.bottomLeft.y >= this.bottomLeft.y &&
               R.topRight.y <= this.topRight.y;
    }
}
```

Question: Implement isBalanced() inside the BangladeshiFlag class using direct access:

```
class BangladeshiFlag {
    Rectangle R;
    Circle C;
    BangladeshiFlag(Rectangle R, Circle C) {
        this.R = R;
        this.C = C;
    }

    boolean isBalanced() {
        // Check if the circle is strictly inside (doesn't touch edges)
        double left = C.center.x - C.radius;
        double right = C.center.x + C.radius;
        double bottom = C.center.y - C.radius;
        double top = C.center.y + C.radius;

        boolean strictlyInside = left > R.bottomLeft.x &&
                                 right < R.topRight.x &&
                                 bottom > R.bottomLeft.y &&
                                 top < R.topRight.y;

        // Check if the circle center is exactly aligned with the rectangle center
        double rectCenterX = (R.bottomLeft.x + R.topRight.x) / 2.0;
        double rectCenterY = (R.bottomLeft.y + R.topRight.y) / 2.0;

        boolean centered = (C.center.x == rectCenterX) && (C.center.y == rectCenterY);

        return strictlyInside && centered;
    }
}
```

Chapter 5: Problem-Solving Scenarios (No Encapsulation)

Here are midterm exam style scenarios designed without private keywords or getters/setters.

Scenario 1: Smart Home Room & Lights

Question: Design a SmartLight and SmartRoom class without encapsulation.
- SmartLight has fields: id (int), brightness (int), and isOn (boolean).
- SmartRoom has roomName (String) and an array of SmartLight objects (max 3).
- Implement addLight(SmartLight light) and double getAverageBrightness() of active lights.

Answer: Implementation using direct field access:

```
class SmartLight {
    int id;
    int brightness;
    boolean isOn;
    SmartLight(int id, int brightness) {
        this.id = id;
        this.brightness = brightness;
        this.isOn = false;
    }
}

class SmartRoom {
    String roomName;
    SmartLight[] lights = new SmartLight[3];
    int count = 0;
    SmartRoom(String roomName) {
        this.roomName = roomName;
    }
    void addLight(SmartLight light) {
        if (count < 3) {
            lights[count] = light;
            count++;
        }
    }
    double getAverageBrightness() {
        int sum = 0, active = 0;
        for (int i = 0; i < count; i++) {
            if (lights[i].isOn) {
                sum += lights[i].brightness;
                active++;
            }
        }
        return active == 0 ? 0.0 : (double) sum / active;
    }
}
```

Scenario 2: Ride-Sharing Nearest Driver Allocation

Question: Design a Passenger and Driver class using direct variable access.
- Passenger has: name, location (Point).
- Driver has: name, location (Point), vehicleType (String), and isAvailable (boolean).
- Write a static method inside Driver:
static Driver findNearest(Driver[] list, Passenger p, String type)
which returns the nearest available driver of the matching vehicleType.

Answer: Driver allocation using direct object variables:

```
class Passenger {
    String name;
    Point location;
    Passenger(String name, Point location) {
        this.name = name;
        this.location = location;
    }
}

class Driver {
    String name;
    Point location;
    String vehicleType;
    boolean isAvailable;
    Driver(String name, Point location, String vehicleType) {
        this.name = name;
        this.location = location;
        this.vehicleType = vehicleType;
        this.isAvailable = true;
    }
    static Driver findNearest(Driver[] list, Passenger p, String type) {
        Driver match = null;
        double min = Double.MAX_VALUE;
        for (int i = 0; i < list.length; i++) {
            Driver d = list[i];
            if (d.isAvailable && d.vehicleType.equalsIgnoreCase(type)) {
                double dist = d.location.distance(p.location);
                if (dist < min) {
                    min = dist;
                    match = d;
                }
            }
        }
        return match;
    }
}
```

Chapter 6: Mock Midterm Exam & Explanations

MOCK EXAM QUESTIONS

Question: 1. Write classes Point and Line using direct variable access.

- **Point has variables x and y.**
- **Line has start and end points.**
- **Implement a method isVertical() inside Line that checks if the line is vertical.**

Question: 2. What is the console output of the following main method?

```
class Test {
    int data = 10;
    static int count = 50;
    Test(int d) {
        data = d;
        count += d;
    }
    void shift(Test other) {
        this.data = other.data + 10;
        count += this.data;
    }
}

// Inside Main class main method:
Test t1 = new Test(5);
Test t2 = new Test(15);
t1.shift(t2);
System.out.println(t1.data);
System.out.println(t2.data);
System.out.println(Test.count);
```

MOCK EXAM SOLUTIONS

Answer: Question 1 (Line & Point):

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

class Line {
    Point start, end;
    Line(Point start, Point end) { this.start = start; this.end = end; }
    boolean isVertical() {
        return this.start.x == this.end.x;
    }
}
```

Answer: Question 2 (Tracing Output):

Step 1: 'Test t1 = new Test(5);' -> t1.data = 5, static count becomes 50 + 5 = 55.

Step 2: 'Test t2 = new Test(15);' -> t2.data = 15, static count becomes 55 + 15 = 70.

Step 3: 't1.shift(t2);' executes on t1:

- other is t2. other.data is 15.
- this.data (t1.data) = other.data + 10 = 15 + 10 = 25.
- static count = count + this.data = 70 + 25 = 95.

Step 4: Prints output:

25
15
95